Traffic Light FSM Controller Design with Pedestrian Crosswalk Buttons

Worcester Polytechnic Institute

ECE 2029 - Introduction to Digital Circuit Design

Professor Koksal Mus

Teremun Beard, Perry Flagg, and Gideon Drury

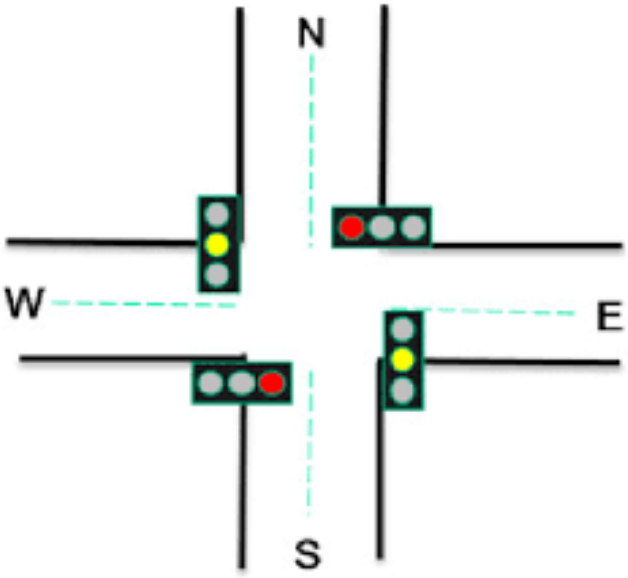

Table of Contents

Problem Statement

Design and implement a finite state machine (FSM) based traffic light controller that manages intersection traffic for North-South and East-West directions. Integrate pedestrian crosswalk functionality that the controller must prioritize, ensuring that pedestrian crossing requests take precedence over regular vehicular traffic, maintaining safety and efficient traffic flow.

Inputs: Clock, Button

States: **Green** (North-South), **Green** (East-West), **Yellow** (North-South), **Yellow** (East-West), **Red=**NOT(green OR yellow) (North-South), **Red=**NOT(green OR yellow) (East-West)

Outputs: Pedestrian Light - North-South, East-West

Originally, we considered having two clocks: one for normal traffic flow, and one for the pedestrian crosswalk and yellow-light timing. The first clock would keep a direction green for 60 seconds before transitioning to yellow, then red, allowing the adjacent traffic light to switch to green.
The second clock would last for 10 seconds and would handle both the pedestrian crosswalk and the yellow light. After careful consideration, we revised our idea and created three distinct timing signals named clk_green, clk_yellow, and clk_crosswalk, corresponding to the green-light interval, yellow-light interval, and pedestrian crosswalk interval. We initially proposed timing values of 63 seconds for green, 7 seconds for yellow, and 21 seconds for the pedestrian signal. After simulation testing, the final values were adjusted to 112 seconds for green, 7 seconds for yellow, and 28 seconds for the crosswalk to prevent the green-light interval from becoming too short after pedestrian requests.
The red light would be determined through combinational logic, where a direction would be red whenever it is not currently green or yellow. These timing values were chosen as they closely resemble realistic intersection timings.

Proposed Solution

The system transitions between states using three timing signals for the green-light, yellow-light, and pedestrian crosswalk intervals.

Inputs:
CLK Moment: (The clocks will not be used as an input. Instead, we will use an input based on when the clock resets. This will be used for all three clocks: Green light, Yellow light, and Pedestrian signal.)
Pedestrian Button (Interrupts the continuous loop to allow pedestrians to cross.)
The following states are:
North-South green light, North-South yellow light
(Both physical North and South lights are activated as one)
East-West green light, East-West yellow light
(Both physical East and West lights are activated as one)
Four pedestrian “walk” indicator signals

(will all activate at once, stopping all traffic.)
*Will assume that pedestrians will observe the walk signal and not press the button again.
Red light: Derived through combinational logic, defined as (Red = NOT(Green or Yellow))
These states will cycle through a loop using individual clocks whose time is based on real-life traffic lights' timers. The system will not automatically go to a crosswalk each cycle, but rather only when a pedestrian requests it through the pedestrian buttons.

Current State	Inputs	Next State	Outputs
000	0XXX	000	10000
000	1XXX	001	10000
001	XX0X	001	01000
001	XX10	010	01000
001	XX11	100	01000
010	0XXX	010	00100
010	1XXX	011	00100
011	XX0X	011	00010
011	XX10	000	00010
011	XX11	101	00010
100	X0XX	100	00001
100	X1XX	010	00001
101	X0XX	101	00001
101	X1XX	000	00001

Table 1: State and Output Table

Within this table, we show how the Moore machine's output is determined through the current state. Each state is within its binary representation in the table as they count up in binary (ie, S0=000, S1=001, S2=010, …). The inputs are one bit for each clock and one bit for the pedestrian button. The X represents a "do not care" within the function, as it does not change which state the next state is. The output is decoded here as the first two bits represent the NS light, the second two represent the EW light, and the last bit represents the crosswalk light.
For the crosswalk, 1 is green, and 0 is red; for the others, 10 is green, 01 is yellow, and 00 is red.

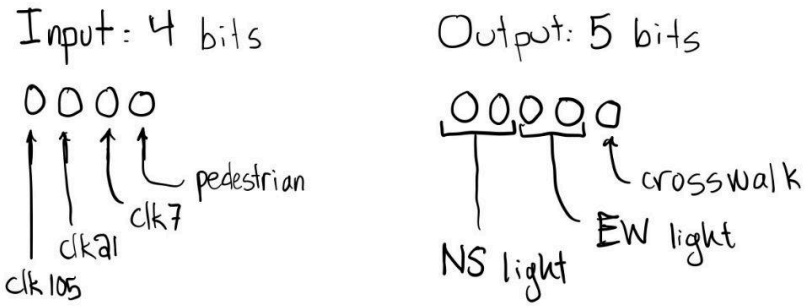

Figure 1: Labeled Binary Inputs and Outputs

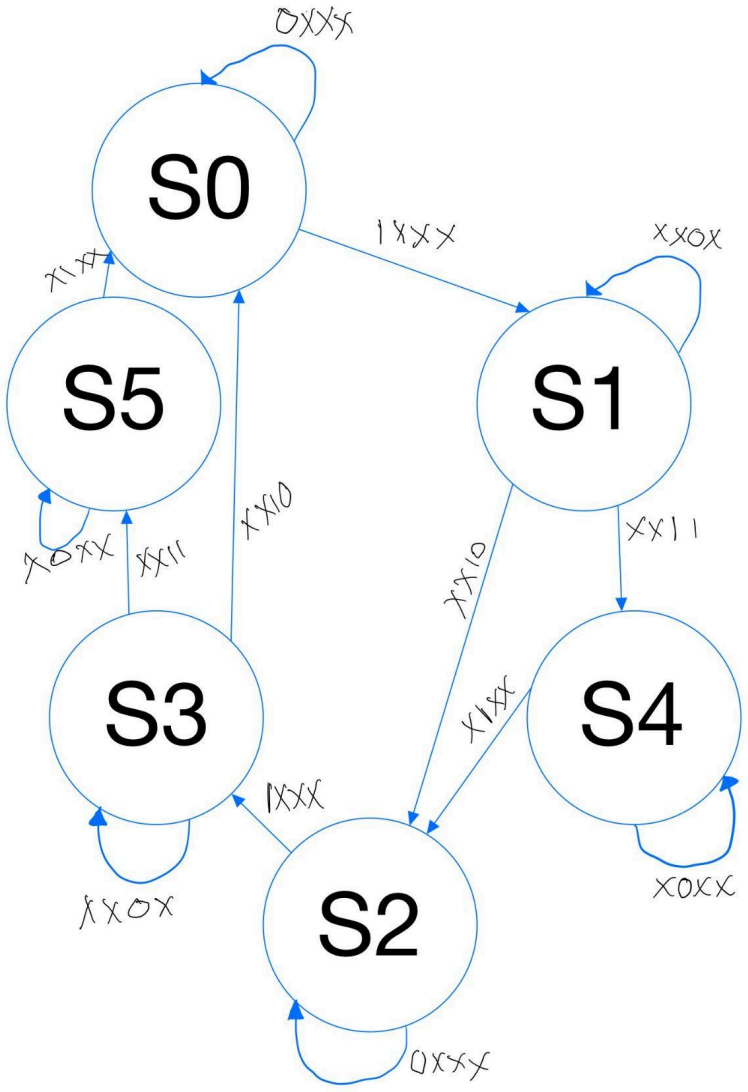

Figure 2: State Diagram of Moore Machine

Code Explanation

The project was programmed in Verilog using Vivado 2024.2.

Within our Vivado code, there are three design modules (clock_gen.v, traffic_FSM, intersection_top_module.v) and a testbench module to run a simulation. We followed a technique used in our prelabs, using our state diagram and table as the main source of influence upon the code, and updated them with solutions as problems occurred. Most of the theory behind our code was derived from the prelabs finished within the term and lab assignments. Particularly, the use of parameters and the "current_state=next_state" was helpful.

clock_gen.v:

```
module clock_module (
   output reg clk_green,
   output reg clk_yellow,
   output reg clk_crosswalk
);

initial begin
   clk_green = 0;
   clk_yellow = 0;
   clk_crosswalk = 0;
end

always begin
   #112 clk_green = ~clk_green;
end

always begin
   #7 clk_yellow = ~clk_yellow;
end

always begin
   #28 clk_crosswalk = ~clk_crosswalk;
end

endmodule
```

Within this module, different timing signals were created for the green, yellow, and pedestrian crosswalk intervals. The green interval uses a delay of 112 nanoseconds, the yellow interval uses a delay of 7 nanoseconds, and the pedestrian crosswalk interval uses a delay of 28 nanoseconds. In the simulation, one nanosecond represents one real-time second for design purposes. The signals were named clk_green, clk_yellow, and clk_crosswalk to describe their function rather than their numeric delay values. We ensured these times were multiples of each other, as they will be running the whole time concurrently.

traffic_FSM.v:

Within this module, we input all three of our clocks from the clock generator, a general clock, a reset button for the system, and the pedestrian crossing button. The only output is the decoded output indicating which light is on. This module follows our state table very closely; however, there were additions made after the first simulation. An issue arose in the first simulation, as the pedestrian button would only register while pressed; thus, if the pedestrian button was not pressed exactly at the moment the yellow light was ending, it would not be considered, and the crosswalk state would be missed. To fix this, we created a new input dependent on the pedestrian button input called pedestrian request, which would hold its signal until acknowledged by the crosswalk state occurring. This pedestrian request would also be brought to zero through the reset input previously stated.

Another adjustment was converting the continuously running clock signals into moment-based transition signals. To remedy this, we created moment-based transition signals from the three timing signals: clk_green_moment, clk_yellow_moment, and clk_crosswalk_moment, which occur only at the instant when a timing signal reaches a transition point.

The bulk of the code came from the second half of traffic_FSM.v through the current state and next state parameters. Setting each state as a parameter (naming it for easier understanding) and then manually listing the state table through code:

```
next_state = current_state;
case (current_state)
    NS_GREEN: if (clk_green_moment) next_state = NS_YELLOW;
    NS_YELLOW: if (clk_yellow_moment) next_state = ped_request ? CROSSWALK_1 : EW_GREEN;
    CROSSWALK_1: if (clk_crosswalk_moment) next_state = EW_GREEN;
    EW_GREEN: if (clk_green_moment) next_state = EW_YELLOW;
    EW_YELLOW: if (clk_yellow_moment) next_state = ped_request ? CROSSWALK_2 : NS_GREEN;
    CROSSWALK_2: if (clk_crosswalk_moment) next_state = NS_GREEN;
```

These states loop seamlessly between each other through the use of our clocks, and once a pedestrian request has arrived at any point between yellow lights, it will be offered at the next opportunity. Through these states, we derived our outputs (lights). Using the reset button would send the current state back to NS_GREEN.

Our finite state machine is a Moore machine; the outputs are determined through the current state of the machine, which makes the outputs easy. Just setting each output to each specific state (the 00001 output is tied to both crosswalk states).

intersection_top_module.v:

```
(input wire clk, input wire reset, input wire ped_button, output wire [4:0] lights);

wire clk_green, clk_yellow, clk_crosswalk;

clock_module clkgen (
```

```
    .clk_green(clk_green),
    .clk_yellow(clk_yellow),
    .clk_crosswalk(clk_crosswalk)
);

traffic_FSM fsm (
    .clk(clk),
    .reset(reset),
    .clk_green(clk_green),
    .clk_yellow(clk_yellow),
    .clk_crosswalk(clk_crosswalk),
    .ped_button(ped_button),
    .lights(lights)
);
```

This final design module ties everything together and prepares it for the simulation and testbench. Drawing in all the important inputs and outputs from the other modules so that they can be tested through the simulation. This module is important, as it connects the clock generator and FSM together. This allows the full design to be simulated through the testbench.

FSM_tb.v:

Our test bench draws from the intersection_top_module.v to know what is important and what to include within the simulation. Then, at “initial begin”, it runs the program in which the testbench applies two sample pedestrian requests at different points in the traffic cycle:

```
initial begin clk = 0; reset = 1; ped_button = 0;

        #5 reset = 0;

        #100 ped_button = 1;
        #10  ped_button = 0;
        #370 ped_button = 1;
        #10  ped_button = 0;

        #1000 $finish;
```

Simulation Results

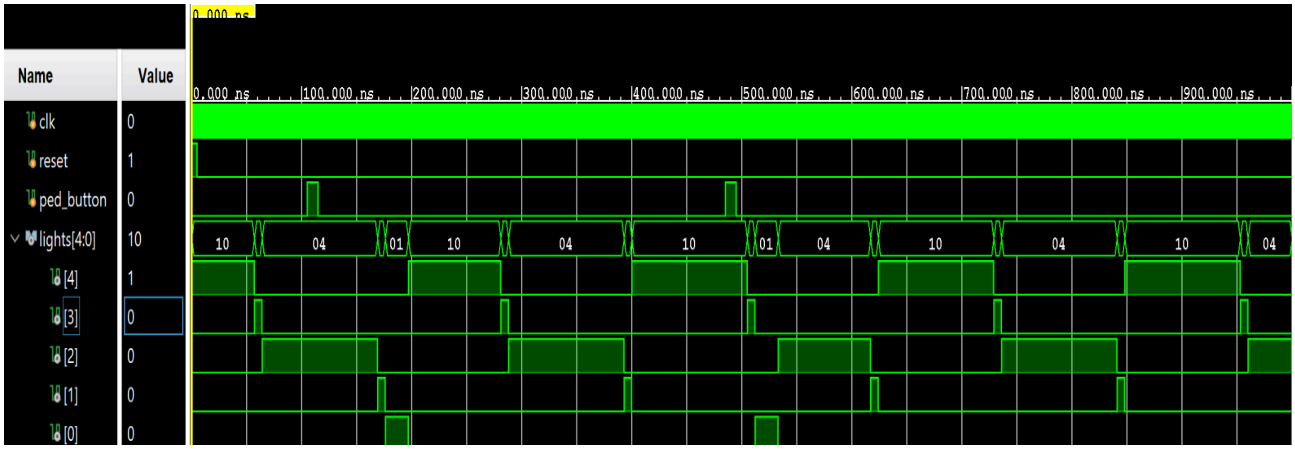

*Figure 3: Simulation Results *(One real-time second equals one nanosecond within the simulation.)*

In Figure 3, we see that the waveform begins with the reset signal active. This is what initializes the traffic light controller and begins the cycle. Before it can complete a full cycle, the pedestrian button is pressed to request the crosswalk. At the next available point, the pedestrian cross request is accepted, which leads to a reduced green light time, but not by a significant amount. This was accounted for in the initial planning and design. The loop then continues through until another pedestrian request occurs shortly before the yellow-light transition, which is also granted at the appropriate time. Overall, the simulation shows that the system correctly prioritizes pedestrian requests when the button is pressed while still maintaining normal traffic flow when no pedestrian request is active.

Conclusion

In total, we used six states to implement the traffic light controller design. The final system controlled North-South and East-West traffic, as well as pedestrian crossing requests using a finite-state machine. At first, we believed that using three continuously running clocks would be enough to control the green, yellow, and crosswalk timing. However, once we began coding in Vivado, we realized that the clocks themselves should not directly control the FSM transitions. Instead, we needed moment-based signals that indicated when each clock had reached the correct transition point.

One issue we encountered was that our original timing values did not line up because the clocks were always running in the background. This caused some yellow-light intervals to be shorter or longer than expected. To fix this, we adjusted the timing values so the green, yellow, and crosswalk periods worked more consistently together. Our final timing values were 112 seconds for green, 7 seconds for yellow, and 28 seconds for the pedestrian crosswalk. To make the code clearer, the timing signals were named by function rather than by their original numeric timing values. The final signals were clk_green, clk_yellow, and clk_crosswalk, corresponding to the green-light, yellow-light, and pedestrian-crosswalk intervals.

Additionally, we found that the pedestrian button signal would only be detected if it was pressed at the exact moment the FSM checked for it. To solve this, we added a pedestrian request signal that stays active until the system reaches the crosswalk state. This ensured that pedestrian requests were not missed and were granted at the next appropriate opportunity. Overall, the final simulation showed that the traffic light controller functioned as intended. The system cycled through the normal traffic light sequence, responded to pedestrian requests, and maintained safe traffic behavior. Although the crosswalk state slightly reduced the next green-light interval, the timing still allowed traffic to continue flowing without making the overall cycle too long.

For future improvements, protected left-turn states could be added for both directions to improve intersection efficiency. Another improvement would be to activate only the specific crosswalk requested, rather than enabling all pedestrian crossings at once. This would allow non-conflicting traffic movements to continue and reduce unnecessary traffic delays.